Day 30 — Final Capstone

INPUT :

```
E101,C001,P001,VIEW,1,2026-10-05T11:00:01
E102,C001,P001,CART,1,2026-10-05T11:00:02
E103,C001,P001,PURCHASE,1,2026-10-05T11:00:03
E104,C002,P002,PURCHASE,1,2026-10-05T11:00:04
E105,C003,P003,PURCHASE,2,2026-10-05T11:00:05
E106,C001,P003,PURCHASE,3,2026-10-05T11:00:06
```

STREAMING SCALA PROGRAM :

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._
import org.apache.spark.storage.StorageLevel

case class StreamEvent(
  eventId: String,
  customerId: String,
  productId: String,
  eventType: String,
  quantity: Int,
  timestamp: String
)

object Day30StreamingApp {

  def main(args: Array[String]): Unit = {

    val conf =
      new SparkConf()
        .setAppName("Day30-Ecommerce-Streaming")
        .setMaster("local[4]")

    val ssc =
      new StreamingContext(
        conf,
        Seconds(5)
```

```
  )

ssc.checkpoint("checkpoint")

val spark =
  SparkSession.builder()
    .config(ssc.sparkContext.getConf)
    .getOrCreate()

import spark.implicits._

println()
println("================================================")
println("      DAY 30 - E-COMMERCE STREAMING")
println("================================================")
println()
println("Listening on localhost:9999")
println()

// ------------------------------------------------
// REFERENCE DATA
// ------------------------------------------------

val products =
  Map(
    "P001" -> ("Laptop", "Electronics", 55000.0),
    "P002" -> ("Smartphone", "Electronics", 30000.0),
    "P003" -> ("Headphones", "Electronics", 2500.0),
    "P004" -> ("Keyboard", "Accessories", 1800.0),
    "P005" -> ("Mouse", "Accessories", 900.0),
    "P006" -> ("Monitor", "Electronics", 15000.0),
    "P007" -> ("Backpack", "Travel", 2200.0),
    "P008" -> ("Shoes", "Fashion", 3500.0)
  )

val customers =
  Map(
    "C001" -> ("Hyderabad", "PREMIUM"),
    "C002" -> ("Delhi", "REGULAR"),
    "C003" -> ("Mumbai", "PREMIUM"),
    "C004" -> ("Bangalore", "REGULAR"),
    "C005" -> ("Chennai", "PREMIUM"),
    "C006" -> ("Pune", "REGULAR")
  )
```

```
val productBroadcast =
  ssc.sparkContext.broadcast(products)

val customerBroadcast =
  ssc.sparkContext.broadcast(customers)

// ------------------------------------------------
// ACCUMULATOR
// ------------------------------------------------

val invalidEvents =
  ssc.sparkContext.longAccumulator(
    "StreamingInvalidEvents"
  )

// ------------------------------------------------
// SOCKET SOURCE
// ------------------------------------------------

val input =
  ssc.socketTextStream(
    "localhost",
    9999
  )

// ------------------------------------------------
// PARSE
// ------------------------------------------------

val parsed =
  input.flatMap { line =>

    val parts =
      line.split(",")

    try {

      if (parts.length != 6) {

        invalidEvents.add(1)

        None
```

```
      } else {

        Some(
          StreamEvent(
            parts(0).trim,
            parts(1).trim,
            parts(2).trim,
            parts(3).trim.toUpperCase,
            parts(4).trim.toInt,
            parts(5).trim
          )
        )
      }

    } catch {

      case _: Exception =>

        invalidEvents.add(1)

        None
    }
  }

val cachedStream =
  parsed.persist(
    StorageLevel.MEMORY_ONLY
  )

// ------------------------------------------------
// CURRENT BATCH
// ------------------------------------------------

cachedStream.foreachRDD { rdd =>

  if (!rdd.isEmpty()) {

    val events =
      rdd.collect()

    println()
    println("-------------------------------------------")
    println("NEW STREAMING BATCH")
    println("-------------------------------------------")
```

```
    events.foreach { event =>

      println(
        s"${event.eventId} | " +
        s"${event.customerId} | " +
        s"${event.productId} | " +
        s"${event.eventType} | " +
        s"${event.quantity}"
      )
    }

    println(
      s"Current batch events = ${rdd.count()}"
    )

    println(
      s"Invalid events = ${invalidEvents.value}"
    )
  }
}

// ------------------------------------------------
// STREAMING PRODUCT QUANTITY
// ------------------------------------------------

val productQuantity =
  cachedStream
    .map(event =>
      (
        event.productId,
        event.quantity
      )
    )
    .reduceByKey(
      (a: Int, b: Int) => a + b
    )

productQuantity.foreachRDD { rdd =>

  if (!rdd.isEmpty()) {

    println()
    println("CURRENT PRODUCT ACTIVITY")
```

```
    println("-----------------------")

    rdd
      .collect()
      .sortBy(_._1)
      .foreach {
        case (product, quantity) =>
          println(
            s"$product -> $quantity units"
          )
      }
  }
}

// ------------------------------------------------
// 30 SECOND WINDOW
// ------------------------------------------------

val windowedPurchases =
  cachedStream
    .filter(
      _.eventType == "PURCHASE"
    )
    .map(event =>
      (
        event.productId,
        event.quantity
      )
    )
    .reduceByKeyAndWindow(
      (a: Int, b: Int) => a + b,
      Seconds(30),
      Seconds(10)
    )

// ------------------------------------------------
// HIGH DEMAND ALERT
// ------------------------------------------------

windowedPurchases.foreachRDD { rdd =>

  val alerts =
    rdd.filter {
      case (_, quantity) =>
```

```
        quantity >= 3
    }

  if (!alerts.isEmpty()) {

    println()
    println("!!! HIGH DEMAND ALERT !!!")
    println("-------------------------")

    alerts
      .collect()
      .foreach {
        case (productId, quantity) =>
          println(
            s"$productId -> " +
            s"$quantity purchases " +
            s"in last 30 seconds"
          )
      }
  }
}

// ------------------------------------------------
// STREAMING SQL
// ------------------------------------------------

cachedStream.foreachRDD { rdd =>

  if (!rdd.isEmpty()) {

    val rows =
      rdd.map { event =>

        val product =
          productBroadcast.value
            .get(event.productId)

        val customer =
          customerBroadcast.value
            .get(event.customerId)

        val productName =
          product.map(_._1)
            .getOrElse("UNKNOWN")
```

```
  val category =
    product.map(_._2)
      .getOrElse("UNKNOWN")

  val price =
    product.map(_._3)
      .getOrElse(0.0)

  val city =
    customer.map(_._1)
      .getOrElse("UNKNOWN")

  val segment =
    customer.map(_._2)
      .getOrElse("UNKNOWN")

  val revenue =
    if (event.eventType == "PURCHASE")
      event.quantity * price
    else
      0.0

  (
    event.eventId,
    event.customerId,
    event.productId,
    event.eventType,
    event.quantity,
    productName,
    category,
    city,
    segment,
    revenue
  )
}
.toDF(
  "eventId",
  "customerId",
  "productId",
  "eventType",
  "quantity",
  "productName",
  "category",
```

```
          "city",
          "segment",
          "revenue"
        )

      rows.createOrReplaceTempView(
        "streaming_events"
      )

      val report =
        spark.sql(
          """
          SELECT
              category,
              COUNT(*) AS events,
              SUM(quantity) AS units,
              ROUND(SUM(revenue), 2) AS revenue
          FROM streaming_events
          WHERE eventType = 'PURCHASE'
          GROUP BY category
          ORDER BY revenue DESC
          """
        )

      println()
      println("STREAMING SQL REPORT")
      println("--------------------")

      report.show(
        truncate = false
      )
    }
  }

  ssc.start()

  ssc.awaitTermination()
 }
}
```

BATCH SCALA PROGRAM :

```
import org.apache.spark.SparkConf
import org.apache.spark.HashPartitioner
import org.apache.spark.sql.{SparkSession, Dataset}
import org.apache.spark.sql.expressions.Window
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._
import org.apache.spark.storage.StorageLevel

case class EcommerceEvent(
  eventId: String,
  customerId: String,
  productId: String,
  eventType: String,
  quantity: Int,
  timestamp: String
)

object Day30BatchApp {

  def main(args: Array[String]): Unit = {

    val conf =
      new SparkConf()
        .setAppName("Day30-Ecommerce-Batch")
        .setMaster("local[4]")

    val spark =
      SparkSession.builder()
        .config(conf)
        .getOrCreate()

    val sc = spark.sparkContext

    import spark.implicits._

    println()
    println("================================================")
    println("       DAY 30 - E-COMMERCE BATCH")
```

```
println("================================================")

// ------------------------------------------------
// 1. RAW RDD
// ------------------------------------------------

val rawEvents =
  sc.textFile("data/events.csv")

println()
println("RAW EVENTS")
println("----------")
println(s"Raw records = ${rawEvents.count()}")

// ------------------------------------------------
// 2. ACCUMULATOR
// ------------------------------------------------

val invalidEventAccumulator =
  sc.longAccumulator("InvalidEventCounter")

// ------------------------------------------------
// 3. CLEAN DATASET
// ------------------------------------------------

val cleanEvents: Dataset[EcommerceEvent] =
  rawEvents.flatMap { line =>

    val parts =
      line.split(",")

    try {

      if (parts.length != 6) {

        invalidEventAccumulator.add(1)

        None

      } else {

        Some(
          EcommerceEvent(
            parts(0).trim,
```

```
            parts(1).trim,
            parts(2).trim,
            parts(3).trim.toUpperCase,
            parts(4).trim.toInt,
            parts(5).trim
          )
        )
      }

    } catch {

      case _: Exception =>

        invalidEventAccumulator.add(1)

        None
    }

  }.toDS()

// ------------------------------------------------
// 4. CACHE / PERSIST
// ------------------------------------------------

val cachedEvents =
  cleanEvents.persist(
    StorageLevel.MEMORY_AND_DISK
  )

println()
println("CLEAN DATA")
println("----------")

cachedEvents.show(
  20,
  truncate = false
)

println(
  s"Invalid events = ${invalidEventAccumulator.value}"
)

// ------------------------------------------------
// 5. PAIR RDD
```

```
// ------------------------------------------------

val eventTypeCounts =
  cachedEvents.rdd
    .map(event =>
      (
        event.eventType,
        1
      )
    )
    .reduceByKey(_ + _)

println()
println("EVENT TYPE COUNTS")
println("-----------------")

eventTypeCounts
  .collect()
  .sortBy(_._1)
  .foreach {
    case (eventType, count) =>
      println(
        s"$eventType -> $count"
      )
  }

// ------------------------------------------------
// 6. PARTITION TUNING
// ------------------------------------------------

val partitionedEvents =
  cachedEvents.rdd
    .map(event =>
      (
        event.productId,
        event.quantity
      )
    )
    .partitionBy(
      new HashPartitioner(4)
    )

println()
println("PARTITION TUNING")
```

```
println("----------------")

println(
  s"Partitions after tuning = " +
  s"${partitionedEvents.getNumPartitions}"
)

// ------------------------------------------------
// 7. PRODUCT DATAFRAME
// ------------------------------------------------

val productSchema =
  StructType(
    Seq(
      StructField(
        "productId",
        StringType,
        false
      ),
      StructField(
        "productName",
        StringType,
        false
      ),
      StructField(
        "category",
        StringType,
        false
      ),
      StructField(
        "price",
        DoubleType,
        false
      )
    )
  )

val productsDF =
  spark.read
    .schema(productSchema)
    .option("header", "false")
    .csv("data/products.csv")

// ------------------------------------------------
```

```
// 8. CUSTOMER DATAFRAME
// ------------------------------------------------

val customerSchema =
  StructType(
    Seq(
      StructField(
        "customerId",
        StringType,
        false
      ),
      StructField(
        "city",
        StringType,
        false
      ),
      StructField(
        "segment",
        StringType,
        false
      )
    )
  )

val customersDF =
  spark.read
    .schema(customerSchema)
    .option("header", "false")
    .csv("data/customers.csv")

// ------------------------------------------------
// 9. BROADCAST
// ------------------------------------------------

val broadcastProducts =
  broadcast(productsDF)

val broadcastCustomers =
  broadcast(customersDF)

// ------------------------------------------------
// 10. JOINS
// ------------------------------------------------
```

```
val enriched =
  cachedEvents
    .toDF()
    .join(
      broadcastProducts,
      Seq("productId"),
      "left"
    )
    .join(
      broadcastCustomers,
      Seq("customerId"),
      "left"
    )

// ------------------------------------------------
// 11. UDF
// ------------------------------------------------

val customerValueUDF =
  udf {
    segment: String =>

      segment match {

        case "PREMIUM" =>
          "HIGH_VALUE"

        case "REGULAR" =>
          "STANDARD_VALUE"

        case _ =>
          "UNKNOWN"
      }
  }

// ------------------------------------------------
// 12. REVENUE
// ------------------------------------------------

val enrichedFinal =
  enriched
    .withColumn(
      "customer_value",
      customerValueUDF(
```

```
      col("segment")
    )
  )
  .withColumn(
    "revenue",
    when(
      col("eventType") === "PURCHASE",
      col("quantity") * col("price")
    ).otherwise(
      lit(0.0)
    )
  )
  .withColumn(
    "event_timestamp",
    to_timestamp(
      col("timestamp")
    )
  )
  .persist(
    StorageLevel.MEMORY_AND_DISK
  )

println()
println("ENRICHED DATA")
println("-------------")

enrichedFinal.show(
  20,
  truncate = false
)

// ------------------------------------------------
// 13. WINDOW FUNCTION
// ------------------------------------------------

val customerWindow =
  Window
    .partitionBy("customerId")
    .orderBy("event_timestamp")
    .rowsBetween(
      Window.unboundedPreceding,
      Window.currentRow
    )
```

```
val runningRevenue =
  enrichedFinal
    .withColumn(
      "running_revenue",
      sum("revenue")
        .over(customerWindow)
    )

println()
println("CUSTOMER RUNNING REVENUE")
println("-----------------------")

runningRevenue
  .select(
    "customerId",
    "eventId",
    "eventType",
    "revenue",
    "running_revenue"
  )
  .orderBy(
    "customerId",
    "event_timestamp"
  )
  .show(
    30,
    truncate = false
  )

// ------------------------------------------------
// 14. PRODUCT AGGREGATION
// ------------------------------------------------

val productSales =
  enrichedFinal
    .filter(
      col("eventType") === "PURCHASE"
    )
    .groupBy(
      "productId",
      "productName",
      "category"
    )
    .agg(
```

```
      sum("quantity")
        .alias("units_sold"),

      sum("revenue")
        .alias("total_revenue")
    )
    .orderBy(
      desc("total_revenue")
    )

println()
println("PRODUCT SALES")
println("-------------")

productSales.show(
  truncate = false
)

// ------------------------------------------------
// 15. CUSTOMER AGGREGATION
// ------------------------------------------------

val customerSales =
  enrichedFinal
    .filter(
      col("eventType") === "PURCHASE"
    )
    .groupBy(
      "customerId",
      "city",
      "segment"
    )
    .agg(
      sum("quantity")
        .alias("items_purchased"),

      sum("revenue")
        .alias("customer_revenue")
    )
    .orderBy(
      desc("customer_revenue")
    )

println()
```

```
println("CUSTOMER SALES")
println("--------------")

customerSales.show(
  truncate = false
)

// ------------------------------------------------
// 16. CATEGORY AGGREGATION
// ------------------------------------------------

val categorySales =
  enrichedFinal
    .filter(
      col("eventType") === "PURCHASE"
    )
    .groupBy(
      "category"
    )
    .agg(
      sum("quantity")
        .alias("units_sold"),

      sum("revenue")
        .alias("revenue")
    )
    .orderBy(
      desc("revenue")
    )

println()
println("CATEGORY SALES")
println("--------------")

categorySales.show(
  truncate = false
)

// ------------------------------------------------
// 17. SPARK SQL
// ------------------------------------------------

enrichedFinal.createOrReplaceTempView(
  "ecommerce_events"
```

```
)

val sqlReport =
  spark.sql(
    """
    SELECT
        category,
        COUNT(*) AS purchase_events,
        SUM(quantity) AS units_sold,
        ROUND(SUM(revenue), 2) AS revenue
    FROM ecommerce_events
    WHERE eventType = 'PURCHASE'
    GROUP BY category
    ORDER BY revenue DESC
    """
  )

println()
println("SPARK SQL REPORT")
println("----------------")

sqlReport.show(
  truncate = false
)

// ------------------------------------------------
// 18. QUERY PLAN / DAG
// ------------------------------------------------

println()
println("EXECUTION PLAN")
println("--------------")

productSales.explain(true)

// ------------------------------------------------
// 19. FINAL SUMMARY
// ------------------------------------------------

val totalEvents =
  cachedEvents.count()

val purchaseEvents =
  enrichedFinal
```

```
    .filter(
      col("eventType") === "PURCHASE"
    )
    .count()

val totalRevenue =
  enrichedFinal
    .agg(
      sum("revenue")
        .alias("total_revenue")
    )
    .collect()
    .head
    .getAs[Double](
      "total_revenue"
    )

println()
println("================================================")
println("          FINAL BATCH SUMMARY")
println("================================================")

println(
  s"Total events       = $totalEvents"
)

println(
  s"Purchase events    = $purchaseEvents"
)

println(
  f"Total revenue      = ₹$totalRevenue%.2f"
)

println(
  s"Invalid events     = " +
  s"${invalidEventAccumulator.value}"
)

println(
  s"Partition count    = " +
  s"${partitionedEvents.getNumPartitions}"
)
```

```
    println("================================================")

    enrichedFinal.unpersist()
    cachedEvents.unpersist()

    spark.stop()
  }
}
```

OUTPUTS :

```
[info] welcome to sbt 1.12.15 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /home/babblu/spark-day30-ecommerce/project
[info] loading settings for project root from build.sbt...
[info] set current project to spark-day30-ecommerce-capstone (in build
file:/home/babblu/spark-day30-ecommerce/)
[info] running (fork) Day30BatchApp
[info] ================================================
[info]        DAY 30 - E-COMMERCE BATCH
[info] ================================================
[info] RAW EVENTS
[info] ----------
[info] Raw records = 15
[info] CLEAN DATA
[info] ----------
[info] +-------+----------+---------+---------+--------+-------------------
[info] |eventId|customerId|productId|eventType|quantity|timestamp
[info] +-------+----------+---------+---------+--------+-------------------
[info] |E001   |C001      |P001     |VIEW     |1       |2026-10-05T10:00:01
[info] |E002   |C001      |P001     |CART     |1       |2026-10-05T10:00:05
[info] |E003   |C001      |P001     |PURCHASE |1       |2026-10-05T10:00:10
[info] |E004   |C002      |P002     |VIEW     |1       |2026-10-05T10:00:15
[info] |E005   |C002      |P002     |CART     |1       |2026-10-05T10:00:20
[info] |E006   |C003      |P003     |PURCHASE |2       |2026-10-05T10:00:25
[info] |E007   |C004      |P004     |VIEW     |1       |2026-10-05T10:00:30
[info] |E008   |C004      |P004     |PURCHASE |2       |2026-10-05T10:00:35
[info] |E009   |C005      |P006     |CART     |1       |2026-10-05T10:00:40
[info] |E010   |C005      |P006     |PURCHASE |1       |2026-10-05T10:00:45
[info] |E011   |C006      |P007     |VIEW     |1       |2026-10-05T10:00:50
[info] |E012   |C006      |P007     |CART     |1       |2026-10-05T10:00:55
[info] |E013   |C001      |P003     |PURCHASE |3       |2026-10-05T10:01:00
[info] |E014   |C003      |P002     |PURCHASE |1       |2026-10-05T10:01:05
[info] |E015   |C005      |P008     |PURCHASE |2       |2026-10-05T10:01:10
[info] +-------+----------+---------+---------+--------+-------------------
```

[info] Invalid events = 0
[info] EVENT TYPE COUNTS
[info] ----------------
[info] CART -> 4
[info] PURCHASE -> 7
[info] VIEW -> 4
[info] PARTITION TUNING
[info] ----------------
[info] Partitions after tuning = 4
[info] ENRICHED DATA
[info] -------------
[info]
+----------+---------+-------+---------+--------+-------------------+-----------+-----------+-------+---------+-------+-------------+-------+------------------+
[info] |customerId|productId|eventId|eventType|quantity|timestamp |productName|category |price |city |segment|customer_value|revenue|event_timestamp |
[info]
+----------+---------+-------+---------+--------+-------------------+-----------+-----------+-------+---------+-------+-------------+-------+------------------+
[info] |C001 |P001 |E001 |VIEW |1 |2026-10-05T10:00:01|Laptop |Electronics|55000.0|Hyderabad|PREMIUM|HIGH_VALUE |0.0 |2026-10-05 10:00:01|
[info] |C001 |P001 |E002 |CART |1 |2026-10-05T10:00:05|Laptop |Electronics|55000.0|Hyderabad|PREMIUM|HIGH_VALUE |0.0 |2026-10-05 10:00:05|
[info] |C001 |P001 |E003 |PURCHASE |1 |2026-10-05T10:00:10|Laptop |Electronics|55000.0|Hyderabad|PREMIUM|HIGH_VALUE |55000.0|2026-10-05 10:00:10|
[info] |C002 |P002 |E004 |VIEW |1 |2026-10-05T10:00:15|Smartphone |Electronics|30000.0|Delhi |REGULAR|STANDARD_VALUE|0.0 |2026-10-05 10:00:15|
[info] |C002 |P002 |E005 |CART |1 |2026-10-05T10:00:20|Smartphone |Electronics|30000.0|Delhi |REGULAR|STANDARD_VALUE|0.0 |2026-10-05 10:00:20|
[info] |C003 |P003 |E006 |PURCHASE |2 |2026-10-05T10:00:25|Headphones |Electronics|2500.0 |Mumbai |PREMIUM|HIGH_VALUE |5000.0 |2026-10-05 10:00:25|
[info] |C004 |P004 |E007 |VIEW |1 |2026-10-05T10:00:30|Keyboard |Accessories|1800.0 |Bangalore|REGULAR|STANDARD_VALUE|0.0 |2026-10-05 10:00:30|
[info] |C004 |P004 |E008 |PURCHASE |2 |2026-10-05T10:00:35|Keyboard |Accessories|1800.0 |Bangalore|REGULAR|STANDARD_VALUE|3600.0 |2026-10-05 10:00:35|
[info] |C005 |P006 |E009 |CART |1 |2026-10-05T10:00:40|Monitor |Electronics|15000.0|Chennai |PREMIUM|HIGH_VALUE |0.0 |2026-10-05 10:00:40|
[info] |C005 |P006 |E010 |PURCHASE |1 |2026-10-05T10:00:45|Monitor |Electronics|15000.0|Chennai |PREMIUM|HIGH_VALUE |15000.0|2026-10-05 10:00:45|
[info] |C006 |P007 |E011 |VIEW |1 |2026-10-05T10:00:50|Backpack |Travel |2200.0 |Pune |REGULAR|STANDARD_VALUE|0.0 |2026-10-05 10:00:50|
[info] |C006 |P007 |E012 |CART |1 |2026-10-05T10:00:55|Backpack |Travel |2200.0 |Pune |REGULAR|STANDARD_VALUE|0.0 |2026-10-05 10:00:55|

```
[info] |C001      |P003     |E013   |PURCHASE |3       |2026-10-05T10:01:00|Headphones
|Electronics|2500.0 |Hyderabad|PREMIUM|HIGH_VALUE    |7500.0 |2026-10-05 10:01:00|
[info] |C003      |P002     |E014   |PURCHASE |1       |2026-10-05T10:01:05|Smartphone
|Electronics|30000.0|Mumbai   |PREMIUM|HIGH_VALUE    |30000.0|2026-10-05 10:01:05|
[info] |C005      |P008     |E015   |PURCHASE |2       |2026-10-05T10:01:10|Shoes      |Fashion
|3500.0 |Chennai  |PREMIUM|HIGH_VALUE    |7000.0 |2026-10-05 10:01:10|
[info]
+----------+---------+-------+---------+--------+------------------+-----------+-----------+-------+---------+-------
+--------------+-------+------------------+
[info] CUSTOMER RUNNING REVENUE
[info] ----------------------
[info] +----------+-------+---------+-------+---------------+
[info] |customerId|eventId|eventType|revenue|running_revenue|
[info] +----------+-------+---------+-------+---------------+
[info] |C001      |E001   |VIEW     |0.0    |0.0            |
[info] |C001      |E002   |CART     |0.0    |0.0            |
[info] |C001      |E003   |PURCHASE |55000.0|55000.0        |
[info] |C001      |E013   |PURCHASE |7500.0 |62500.0        |
[info] |C002      |E004   |VIEW     |0.0    |0.0            |
[info] |C002      |E005   |CART     |0.0    |0.0            |
[info] |C003      |E006   |PURCHASE |5000.0 |5000.0         |
[info] |C003      |E014   |PURCHASE |30000.0|35000.0        |
[info] |C004      |E007   |VIEW     |0.0    |0.0            |
[info] |C004      |E008   |PURCHASE |3600.0 |3600.0         |
[info] |C005      |E009   |CART     |0.0    |0.0            |
[info] |C005      |E010   |PURCHASE |15000.0|15000.0        |
[info] |C005      |E015   |PURCHASE |7000.0 |22000.0        |
[info] |C006      |E011   |VIEW     |0.0    |0.0            |
[info] |C006      |E012   |CART     |0.0    |0.0            |
[info] +----------+-------+---------+-------+---------------+
[info] PRODUCT SALES
[info] -------------
[info] +---------+-----------+-----------+----------+-------------+
[info] |productId|productName|category   |units_sold|total_revenue|
[info] +---------+-----------+-----------+----------+-------------+
[info] |P001     |Laptop     |Electronics|1         |55000.0      |
[info] |P002     |Smartphone |Electronics|1         |30000.0      |
[info] |P006     |Monitor    |Electronics|1         |15000.0      |
[info] |P003     |Headphones |Electronics|5         |12500.0      |
[info] |P008     |Shoes      |Fashion    |2         |7000.0       |
[info] |P004     |Keyboard   |Accessories|2         |3600.0       |
[info] +---------+-----------+-----------+----------+-------------+
[info] CUSTOMER SALES
[info] --------------
```

```
[info] +----------+---------+-------+--------------+---------------+
[info] |customerId|city     |segment|items_purchased|customer_revenue|
[info] +----------+---------+-------+--------------+---------------+
[info] |C001      |Hyderabad|PREMIUM|4             |62500.0        |
[info] |C003      |Mumbai   |PREMIUM|3             |35000.0        |
[info] |C005      |Chennai  |PREMIUM|3             |22000.0        |
[info] |C004      |Bangalore|REGULAR|2             |3600.0         |
[info] +----------+---------+-------+--------------+---------------+
[info] CATEGORY SALES
[info] --------------
[info] +-----------+----------+--------+
[info] |category   |units_sold|revenue |
[info] +-----------+----------+--------+
[info] |Electronics|8         |112500.0|
[info] |Fashion    |2         |7000.0  |
[info] |Accessories|2         |3600.0  |
[info] +-----------+----------+--------+
[info] SPARK SQL REPORT
[info] ----------------
[info] +-----------+--------------+----------+--------+
[info] |category   |purchase_events|units_sold|revenue |
[info] +-----------+--------------+----------+--------+
[info] |Electronics|5             |8         |112500.0|
[info] |Fashion    |1             |2         |7000.0  |
[info] |Accessories|1             |2         |3600.0  |
[info] +-----------+--------------+----------+--------+
[info] EXECUTION PLAN
[info] --------------
[info] == Parsed Logical Plan ==
[info] 'Sort ['total_revenue DESC NULLS LAST], true
[info] +- Aggregate [productId#9, productName#317, category#318], [productId#9,
productName#317, category#318, sum(quantity#11) AS units_sold#1348L, sum(revenue#376)
AS total_revenue#1350]
[info]    +- Filter (eventType#10 = PURCHASE)
[info]       +- Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11,
timestamp#12, productName#317, category#318, price#319, city#325, segment#326,
customer_value#360, revenue#376, to_timestamp(timestamp#12, None, TimestampType,
Some(Asia/Calcutta), false) AS event_timestamp#390]
[info]          +- Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11,
timestamp#12, productName#317, category#318, price#319, city#325, segment#326,
customer_value#360, CASE WHEN (eventType#10 = PURCHASE) THEN (cast(quantity#11 as
double) * price#319) ELSE 0.0 END AS revenue#376]
```

[info] +- Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319, city#325, segment#326, UDF(segment#326) AS customer_value#360]
[info] +- Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319, city#325, segment#326]
[info] +- Join LeftOuter, (customerId#8 = customerId#324)
[info] :- Project [productId#9, eventId#7, customerId#8, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319]
[info] : +- Join LeftOuter, (productId#9 = productId#316)
[info] : :- SerializeFromObject [staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventId, true, false, true) AS eventId#7, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).customerId, true, false, true) AS customerId#8, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).productId, true, false, true) AS productId#9, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventType, true, false, true) AS eventType#10, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).quantity AS quantity#11, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).timestamp, true, false, true) AS timestamp#12]
[info] : : +- ExternalRDD [obj#6]
[info] : +- ResolvedHint (strategy=broadcast)
[info] : +- Relation [productId#316,productName#317,category#318,price#319] csv
[info] +- ResolvedHint (strategy=broadcast)
[info] +- Relation [customerId#324,city#325,segment#326] csv
[info] == Analyzed Logical Plan ==
[info] productId: string, productName: string, category: string, units_sold: bigint, total_revenue: double
[info] Sort [total_revenue#1350 DESC NULLS LAST], true
[info] +- Aggregate [productId#9, productName#317, category#318], [productId#9, productName#317, category#318, sum(quantity#11) AS units_sold#1348L, sum(revenue#376) AS total_revenue#1350]
[info] +- Filter (eventType#10 = PURCHASE)
[info] +- Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319, city#325, segment#326, customer_value#360, revenue#376, to_timestamp(timestamp#12, None, TimestampType, Some(Asia/Calcutta), false) AS event_timestamp#390]
[info] +- Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319, city#325, segment#326, customer_value#360, CASE WHEN (eventType#10 = PURCHASE) THEN (cast(quantity#11 as double) * price#319) ELSE 0.0 END AS revenue#376]

```
[info]            +- Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11,
timestamp#12, productName#317, category#318, price#319, city#325, segment#326,
UDF(segment#326) AS customer_value#360]
[info]               +- Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11,
timestamp#12, productName#317, category#318, price#319, city#325, segment#326]
[info]                  +- Join LeftOuter, (customerId#8 = customerId#324)
[info]                     :- Project [productId#9, eventId#7, customerId#8, eventType#10, quantity#11,
timestamp#12, productName#317, category#318, price#319]
[info]                     :  +- Join LeftOuter, (productId#9 = productId#316)
[info]                     :     :- SerializeFromObject [staticinvoke(class
org.apache.spark.unsafe.types.UTF8String, StringType, fromString,
knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventId, true, false, true) AS
eventId#7, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType,
fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).customerId, true, false,
true) AS customerId#8, staticinvoke(class org.apache.spark.unsafe.types.UTF8String,
StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).productId,
true, false, true) AS productId#9, staticinvoke(class org.apache.spark.unsafe.types.UTF8String,
StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventType,
true, false, true) AS eventType#10, knownnotnull(assertnotnull(input[0, EcommerceEvent,
true])).quantity AS quantity#11, staticinvoke(class org.apache.spark.unsafe.types.UTF8String,
StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).timestamp,
true, false, true) AS timestamp#12]
[info]                     :     :  +- ExternalRDD [obj#6]
[info]                     :     +- ResolvedHint (strategy=broadcast)
[info]                     :        +- Relation [productId#316,productName#317,category#318,price#319]
csv
[info]                     +- ResolvedHint (strategy=broadcast)
[info]                        +- Relation [customerId#324,city#325,segment#326] csv
[info] == Optimized Logical Plan ==
[info] Sort [total_revenue#1350 DESC NULLS LAST], true
[info] +- Aggregate [productId#9, productName#317, category#318], [productId#9,
productName#317, category#318, sum(quantity#11) AS units_sold#1348L, sum(revenue#376)
AS total_revenue#1350]
[info]    +- Project [productId#9, quantity#11, productName#317, category#318, revenue#376]
[info]       +- Filter (isnotnull(eventType#10) AND (eventType#10 = PURCHASE))
[info]          +- InMemoryRelation [customerId#8, productId#9, eventId#7, eventType#10,
quantity#11, timestamp#12, productName#317, category#318, price#319, city#325,
segment#326, customer_value#360, revenue#376, event_timestamp#390], StorageLevel(disk,
memory, deserialized, 1 replicas)
[info]                +- AdaptiveSparkPlan isFinalPlan=true
[info]                   +- == Final Plan ==
[info]                      *(3) Project [customerId#8, productId#9, eventId#7, eventType#10,
quantity#11, timestamp#12, productName#317, category#318, price#319, city#325,
segment#326, UDF(segment#326) AS customer_value#360, CASE WHEN (eventType#10 =
```

PURCHASE) THEN (cast(quantity#11 as double) * price#319) ELSE 0.0 END AS revenue#376, cast(timestamp#12 as timestamp) AS event_timestamp#390]
[info] +- *(3) BroadcastHashJoin [customerId#8], [customerId#324], LeftOuter, BuildRight, false
[info] :- *(3) Project [productId#9, eventId#7, customerId#8, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319]
[info] : +- *(3) BroadcastHashJoin [productId#9], [productId#316], LeftOuter, BuildRight, false
[info] : :- TableCacheQueryStage 0
[info] : : +- InMemoryTableScan [eventId#7, customerId#8, productId#9, eventType#10, quantity#11, timestamp#12]
[info] : : +- InMemoryRelation [eventId#7, customerId#8, productId#9, eventType#10, quantity#11, timestamp#12], StorageLevel(disk, memory, deserialized, 1 replicas)
[info] : : +- *(1) SerializeFromObject [staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventId, true, false, true) AS eventId#7, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).customerId, true, false, true) AS customerId#8, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).productId, true, false, true) AS productId#9, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventType, true, false, true) AS eventType#10, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).quantity AS quantity#11, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).timestamp, true, false, true) AS timestamp#12]
[info] : : +- Scan[obj#6]
[info] : +- BroadcastQueryStage 1
[info] : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=107]
[info] : +- *(1) Filter isnotnull(productId#316)
[info] : +- FileScan csv [productId#316,productName#317,category#318,price#319] Batched: false, DataFilters: [isnotnull(productId#316)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/babblu/spark-day30-ecommerce/data/products.csv], PartitionFilters: [], PushedFilters: [IsNotNull(productId)], ReadSchema: struct<productId:string,productName:string,category:string,price:double>
[info] +- BroadcastQueryStage 2
[info] +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=123]
[info] +- *(2) Filter isnotnull(customerId#324)
[info] +- FileScan csv [customerId#324,city#325,segment#326] Batched: false, DataFilters: [isnotnull(customerId#324)], Format: CSV, Location: InMemoryFileIndex(1

paths)[file:/home/babblu/spark-day30-ecommerce/data/customers.csv], PartitionFilters: [], PushedFilters: [IsNotNull(customerId)], ReadSchema: struct<customerId:string,city:string,segment:string>
[info] +- == Initial Plan ==
[info] Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319, city#325, segment#326, UDF(segment#326) AS customer_value#360, CASE WHEN (eventType#10 = PURCHASE) THEN (cast(quantity#11 as double) * price#319) ELSE 0.0 END AS revenue#376, cast(timestamp#12 as timestamp) AS event_timestamp#390]
[info] +- BroadcastHashJoin [customerId#8], [customerId#324], LeftOuter, BuildRight, false
[info] :- Project [productId#9, eventId#7, customerId#8, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319]
[info] : +- BroadcastHashJoin [productId#9], [productId#316], LeftOuter, BuildRight, false
[info] : :- InMemoryTableScan [eventId#7, customerId#8, productId#9, eventType#10, quantity#11, timestamp#12]
[info] : : +- InMemoryRelation [eventId#7, customerId#8, productId#9, eventType#10, quantity#11, timestamp#12], StorageLevel(disk, memory, deserialized, 1 replicas)
[info] : : +- *(1) SerializeFromObject [staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventId, true, false, true) AS eventId#7, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).customerId, true, false, true) AS customerId#8, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).productId, true, false, true) AS productId#9, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventType, true, false, true) AS eventType#10, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).quantity AS quantity#11, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).timestamp, true, false, true) AS timestamp#12]
[info] : : +- Scan[obj#6]
[info] : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=80]
[info] : +- Filter isnotnull(productId#316)
[info] : +- FileScan csv [productId#316,productName#317,category#318,price#319] Batched: false, DataFilters: [isnotnull(productId#316)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/babblu/spark-day30-ecommerce/data/products.csv], PartitionFilters: [], PushedFilters: [IsNotNull(productId)], ReadSchema: struct<productId:string,productName:string,category:string,price:double>

```
[info]                        +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string,
false]),false), [plan_id=84]
[info]                           +- Filter isnotnull(customerId#324)
[info]                              +- FileScan csv [customerId#324,city#325,segment#326] Batched:
false, DataFilters: [isnotnull(customerId#324)], Format: CSV, Location: InMemoryFileIndex(1
paths)[file:/home/babblu/spark-day30-ecommerce/data/customers.csv], PartitionFilters: [],
PushedFilters: [IsNotNull(customerId)], ReadSchema:
struct<customerId:string,city:string,segment:string>
[info] == Physical Plan ==
[info] AdaptiveSparkPlan isFinalPlan=false
[info] +- Sort [total_revenue#1350 DESC NULLS LAST], true, 0
[info]    +- Exchange rangepartitioning(total_revenue#1350 DESC NULLS LAST, 200),
ENSURE_REQUIREMENTS, [plan_id=685]
[info]       +- HashAggregate(keys=[productId#9, productName#317, category#318],
functions=[sum(quantity#11), sum(revenue#376)], output=[productId#9, productName#317,
category#318, units_sold#1348L, total_revenue#1350])
[info]          +- Exchange hashpartitioning(productId#9, productName#317, category#318, 200),
ENSURE_REQUIREMENTS, [plan_id=682]
[info]             +- HashAggregate(keys=[productId#9, productName#317, category#318],
functions=[partial_sum(quantity#11), partial_sum(revenue#376)], output=[productId#9,
productName#317, category#318, sum#1583L, sum#1584])
[info]                +- Project [productId#9, quantity#11, productName#317, category#318,
revenue#376]
[info]                   +- Filter (isnotnull(eventType#10) AND (eventType#10 = PURCHASE))
[info]                      +- InMemoryTableScan [category#318, eventType#10, productId#9,
productName#317, quantity#11, revenue#376], [isnotnull(eventType#10), (eventType#10 =
PURCHASE)]
[info]                            +- InMemoryRelation [customerId#8, productId#9, eventId#7,
eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319,
city#325, segment#326, customer_value#360, revenue#376, event_timestamp#390],
StorageLevel(disk, memory, deserialized, 1 replicas)
[info]                               +- AdaptiveSparkPlan isFinalPlan=true
[info]                                  +- == Final Plan ==
[info]                                     *(3) Project [customerId#8, productId#9, eventId#7,
eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319,
city#325, segment#326, UDF(segment#326) AS customer_value#360, CASE WHEN
(eventType#10 = PURCHASE) THEN (cast(quantity#11 as double) * price#319) ELSE 0.0 END
AS revenue#376, cast(timestamp#12 as timestamp) AS event_timestamp#390]
[info]                                     +- *(3) BroadcastHashJoin [customerId#8], [customerId#324],
LeftOuter, BuildRight, false
[info]                                        :- *(3) Project [productId#9, eventId#7, customerId#8,
eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319]
[info]                                        :  +- *(3) BroadcastHashJoin [productId#9], [productId#316],
LeftOuter, BuildRight, false
```

[info] : :- TableCacheQueryStage 0
[info] : : +- InMemoryTableScan [eventId#7, customerId#8, productId#9, eventType#10, quantity#11, timestamp#12]
[info] : : +- InMemoryRelation [eventId#7, customerId#8, productId#9, eventType#10, quantity#11, timestamp#12], StorageLevel(disk, memory, deserialized, 1 replicas)
[info] : : +- *(1) SerializeFromObject [staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventId, true, false, true) AS eventId#7, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).customerId, true, false, true) AS customerId#8, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).productId, true, false, true) AS productId#9, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventType, true, false, true) AS eventType#10, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).quantity AS quantity#11, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).timestamp, true, false, true) AS timestamp#12]
[info] : : +- Scan[obj#6]
[info] : +- BroadcastQueryStage 1
[info] : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=107]
[info] : +- *(1) Filter isnotnull(productId#316)
[info] : +- FileScan csv [productId#316,productName#317,category#318,price#319] Batched: false, DataFilters: [isnotnull(productId#316)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/babblu/spark-day30-ecommerce/data/products.csv], PartitionFilters: [], PushedFilters: [IsNotNull(productId)], ReadSchema: struct<productId:string,productName:string,category:string,price:double>
[info] +- BroadcastQueryStage 2
[info] +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=123
[info] +- *(2) Filter isnotnull(customerId#324)
[info] +- FileScan csv [customerId#324,city#325,segment#326] Batched: false, DataFilters: [isnotnull(customerId#324)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/babblu/spark-day30-ecommerce/data/customers.csv], PartitionFilters: [], PushedFilters: [IsNotNull(customerId)], ReadSchema: struct<customerId:string,city:string,segment:string>
[info] +- == Initial Plan ==
[info] Project [customerId#8, productId#9, eventId#7, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319, city#325, segment#326, UDF(segment#326) AS customer_value#360, CASE WHEN (eventType#10 =

PURCHASE) THEN (cast(quantity#11 as double) * price#319) ELSE 0.0 END AS revenue#376, cast(timestamp#12 as timestamp) AS event_timestamp#390]
[info] +- BroadcastHashJoin [customerId#8], [customerId#324], LeftOuter, BuildRight, false
[info] :- Project [productId#9, eventId#7, customerId#8, eventType#10, quantity#11, timestamp#12, productName#317, category#318, price#319]
[info] : +- BroadcastHashJoin [productId#9], [productId#316], LeftOuter, BuildRight, false
[info] : :- InMemoryTableScan [eventId#7, customerId#8, productId#9, eventType#10, quantity#11, timestamp#12]
[info] : : +- InMemoryRelation [eventId#7, customerId#8, productId#9, eventType#10, quantity#11, timestamp#12], StorageLevel(disk, memory, deserialized, 1 replicas)
[info] : : +- *(1) SerializeFromObject [staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventId, true, false, true) AS eventId#7, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).customerId, true, false, true) AS customerId#8, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).productId, true, false, true) AS productId#9, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).eventType, true, false, true) AS eventType#10, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).quantity AS quantity#11, staticinvoke(class org.apache.spark.unsafe.types.UTF8String, StringType, fromString, knownnotnull(assertnotnull(input[0, EcommerceEvent, true])).timestamp, true, false, true) AS timestamp#12]
[info] : : +- Scan[obj#6]
[info] : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=80]
[info] : +- Filter isnotnull(productId#316)
[info] : +- FileScan csv [productId#316,productName#317,category#318,price#319] Batched: false, DataFilters: [isnotnull(productId#316)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/babblu/spark-day30-ecommerce/data/products.csv], PartitionFilters: [], PushedFilters: [IsNotNull(productId)], ReadSchema: struct<productId:string,productName:string,category:string,price:double>
[info] +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=84]
[info] +- Filter isnotnull(customerId#324)
[info] +- FileScan csv [customerId#324,city#325,segment#326] Batched: false, DataFilters: [isnotnull(customerId#324)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/home/babblu/spark-day30-ecommerce/data/customers.csv], PartitionFilters: [], PushedFilters: [IsNotNull(customerId)], ReadSchema: struct<customerId:string,city:string,segment:string>

```
[info] ================================================
[info]          FINAL BATCH SUMMARY
[info] ================================================
[info] Total events       = 15
[info] Purchase events    = 7
[info] Total revenue      = ₹123100.00
[info] Invalid events     = 0
[info] Partition count    = 4
[info] ================================================
[success] Total time: 10 s, completed Oct 5, 2026, 12:31:34 PM
babblu@LAPTOP-E7CK2BER:~/spark-day30-ecommerce$ nano
src/main/scala/Day30StreamingApp.scala
babblu@LAPTOP-E7CK2BER:~/spark-day30-ecommerce$ cd ~/spark-day30-ecommerce
sbt compile
[info] welcome to sbt 1.12.15 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /home/babblu/spark-day30-ecommerce/project
[info] loading settings for project root from build.sbt...
[info] set current project to spark-day30-ecommerce-capstone (in build
file:/home/babblu/spark-day30-ecommerce/)
[info] Executing in batch mode. For better performance use sbt's shell
[info] compiling 1 Scala source to
/home/babblu/spark-day30-ecommerce/target/scala-2.12/classes ...
[warn] one deprecation (since Spark 3.4.0); re-run with -deprecation for details
[warn] one warning found
[success] Total time: 5 s, completed Oct 5, 2026, 12:31:55 PM
babblu@LAPTOP-E7CK2BER:~/spark-day30-ecommerce$ cd ~/spark-day30-ecommerce

SBT_OPTS="-Dlog4j.configurationFile=$(pwd)/src/main/resources/log4j2.properties" \
sbt "runMain Day30StreamingApp"
[info] welcome to sbt 1.12.15 (Ubuntu Java 17.0.20.1)
[info] loading project definition from /home/babblu/spark-day30-ecommerce/project
[info] loading settings for project root from build.sbt...
[info] set current project to spark-day30-ecommerce-capstone (in build
file:/home/babblu/spark-day30-ecommerce/)
[info] running (fork) Day30StreamingApp
[info] ================================================
[info]       DAY 30 - E-COMMERCE STREAMING
[info] ================================================
[info] Listening on localhost:9999
[info] -------------------------------------------
[info] NEW STREAMING BATCH
[info] -------------------------------------------
[info] E101 | C001 | P001 | VIEW | 1
[info] E102 | C001 | P001 | CART | 1
```

```
[info] E103 | C001 | P001 | PURCHASE | 1
[info] E104 | C002 | P002 | PURCHASE | 1
[info] E105 | C003 | P003 | PURCHASE | 2
[info] E106 | C001 | P003 | PURCHASE | 3
[info] Current batch events = 6
[info] Invalid events = 0
[info] CURRENT PRODUCT ACTIVITY
[info] ----------------------
[info] P001 -> 3 units
[info] P002 -> 1 units
[info] P003 -> 5 units
[info] STREAMING SQL REPORT
[info] -------------------
[info] +-----------+------+-----+-------+
[info] |category   |events|units|revenue|
[info] +-----------+------+-----+-------+
[info] |Electronics|4     |7    |97500.0|
[info] +-----------+------+-----+-------+
[info] !!! HIGH DEMAND ALERT !!!
[info] ------------------------
[info] P003 -> 5 purchases in last 30 seconds
[info] !!! HIGH DEMAND ALERT !!!
[info] ------------------------
[info] P003 -> 5 purchases in last 30 seconds
[info] !!! HIGH DEMAND ALERT !!!
[info] ------------------------
[info] P003 -> 5 purchases in last 30 seconds
```